PARIS PANTHÉON-ASSAS UNIVERSITÉ

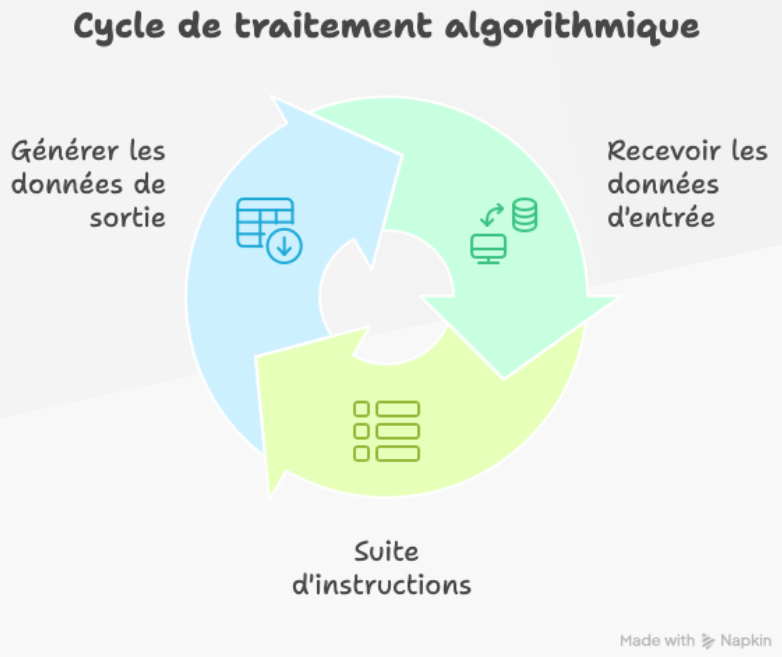
# TI205 - Introduction à la complexité

## CM1 - Notions de base

rado.rakotonarivo@efrei.fr

# Rappel sur les algorithmes

- Un algorithme est une suite finie d'instructions explicites, précises, non-ambigues permettant de résoudre un problème donné.
- La majorité des algorithmes transforment des données en entrée en des données en sortie.



# Analyse d'algorithmes

- L'analyse d'algorithmes est une discipline qui étudie l'**efficacité** des algorithmes.
- L'efficacité d'un algorithme peut être mesurée par :
  - Le temps d'exécution
  - L'espace mémoire utilisé

**Pourquoi ?**
Mesurer l'efficacité d'un algorithme permet de choisir le meilleur algorithme pour un problème donné.

# Temps d'exécution

- Le *temps d'exécution* d'un algorithme est le temps nécessaire pour exécuter cet algorithme.
- Le plus souvent ce temps grandit avec la taille des données en entrée.
- Afin de définir le temps d'exécution, on étudie souvent le **pire cas** (le cas qui prend le plus de temps):
  - Plus facile à déterminer
  - Plus pertinent pour les applications critiques

**Exemple :** Le pire cas pour un algorithme de recherche dans un tableau est quand l'élément recherché est n'est pas présent dans le tableau.

# Espace mémoire

- L'*espace mémoire* utilisé par un algorithme est la quantité de mémoire nécessaire pour exécuter cet algorithme.
- L'espace mémoire utilisé par un algorithme peut être mesuré par :
  - La taille des données en entrée
  - La taille des données en sortie
  - La taille des données intermédiaires

> **Note :** Dans ce cours nous nous intéresserons principalement au temps d'exécution des algorithmes.

# Complexité des algorithmes

- La complexité d'un algorithme est **une mesure de l'efficacité** de cet algorithme.
- La complexité d'un algorithme peut être mesurée par :
  - La complexité en temps qui représente le temps d'exécution de l'algorithme
  - La complexité en espace qui représente l'espace mémoire utilisé par l'algorithme

# Complexité en temps

**Définition :**
La complexité en temps d'un algorithme est une *fonction* qui donne le temps d'exécution de cet algorithme en fonction de la taille des entrées.

▶️ Elle indique comment le temps d'exécution de l'algorithme évolue en fonction de la taille des données en entrée.
▶️ On s'intéresse souvent à la **complexité en temps dans le pire cas**.

# Comment mesurer la complexité en temps d'un algorithme ?

## Études expérimentales

1. Implanter l'algorithme dans un langage de programmation
2. L'exécuter avec des entrées de différentes tailles
3. Mesurer le temps d'exécution avec des entrées de différentes tailles.
4. Estimer une fonction qui donne le temps d'exécution de l'algorithme en fonction de la taille des données en entrée.

# Comment mesurer la complexité en temps d'un algorithme ?

## Limites des études expérimentales

- Cette méthode est imprécise et dépend de la machine sur laquelle l'algorithme est exécuté.
- L'exécution de l'algorithme peut être très longue pour des entrées de grande taille.
- L'algorithme peut ne pas être correctement implanté.

# Comment mesurer la complexité en temps d'un algorithme ?

Il faudrait trouver une méthode qui ne dépend ni de la machine sur laquelle l'algorithme est exécuté, ni du langage dans lequel l'algorithme est implanté.

## Analyse théorique

- Elle permet d'évaluer le temps d'exécution d'un algorithme sans l'exécuter (pas de dépendance de la machine ou du langage).
- Utilise souvent une description formelle de l'algorithme (pseudo-code, langage naturel) au lieu d'une implantation.
- Elle permet de prendre en compte tous les cas possibles (toutes les tailles de données d'entrées).

# Analyse théorique

- Pour mesurer le temps d'exécution d'un algorithme, on compte le nombre d'opérations élémentaires effectuées par l'algorithme.
- On considère alors dans ce cas:
  - Les affectations
  - Les comparaisons
  - Les opérations arithmétiques
  - Les appels de fonctions
  - Les boucles
  - Les conditions

# Exemple 1

On considère l'algorithme suivant qui calcule la somme des éléments d'un tableau de $n$ éléments.

```
Function: sum_array (T : array of integers, n: integer) → integer
Local variables:
    s : integer
    i : integer
begin
    s ← 0
    for i ← 1 to n do
        s ← s + T[i]
    end for
    return s
end
```

# Exemple 1

Comptons le nombre d'opérations élémentaires effectuées par la fonction `sum_array`.

```
Function: sum_array (T : array of integers, n: integer) → integer
Local variables:
    s : integer
    i : integer
begin
    s ← 0                                # 1 opération
    for i ← 1 to n do                    # 2 opérations (initialisation de i et comparaison i < n)
        s ← s + T[i]                     # 2 opérations (addition et affectation)
    end for                              # 3 opérations (incrémentation de i et comparaison i < n)
    return s                             # 1 opération
end
```

- Les instructions dans le corps de la boucle sont répétées $n$ fois.
- On a $1 + 2 + n(2 + 3) + 1 = 5n + 4$ opérations élémentaires.
- Si on note $T(n)$ le nombre d'opérations élémentaires de la fonction `sum_array`, on a $T(n) = 5n + 4$.

# Expression de la complexité en temps

- Lors de l'étude de la complexité en temps d'un algorithme, on cherche à déterminer comment le temps d'exécution de l'algorithme évolue en fonction de la taille des données en entrée.
- On s'intéresse à de grandes tailles de données en entrée car les petites tailles d'entrées ne sont pas significatives.
- Pour cela, on utilise une *notation mathématique* afin d'exprimer l'**ordre de grandeur** du temps d'exécution de l'algorithme.

# Notation $\mathcal{O}$ (se lit "grand O")

La notation $\mathcal{O}$ est une notation mathématique qui permet de donner un ordre de grandeur d'une fonction.

**Définition :**
Soient $f$ et $g$ deux fonctions de $\mathbb{N}$ dans $\mathbb{R}^+$. On dit que $f(n) \in \mathcal{O}(g(n))$ si et seulement s'il existe deux constantes $c$ et $n_0$ telles que pour tout $n \geq n_0$ on a $f(n) \leq c \cdot g(n)$.

On peut comprendre cette définition comme suit: pour $n$ suffisamment grand, la fonction $f$ est bornée par la fonction $g$, c'est à dire que $f$ ne croit pas plus vite que $g$.

# Règles de la notation $\mathcal{O}$

- Si $f(n) \in \mathcal{O}(g(n))$, alors on peut dire que $f(n) = \mathcal{O}(g(n))$ ou $f(n)$ est $\mathcal{O}(g(n))$.
- Si $f(n)$ est un polynôme de degré $d$, alors $f(n) \in \mathcal{O}(n^d)$.
  - On peut donc supprimer les termes de plus bas degré que $d$.
  - On peut supprimer les facteurs constants.
- En pratique, on utilise la *classe* la plus basse possible pour $f(n)$.
  - On dit que $2n \in \mathcal{O}(n)$ même si on a aussi $2n \in \mathcal{O}(n^2)$.
- On utilise l'expression la plus simple possible
  - On dit que $3n + 5 \in \mathcal{O}(n)$ même si on a aussi $3n + 5 \in \mathcal{O}(3n)$.

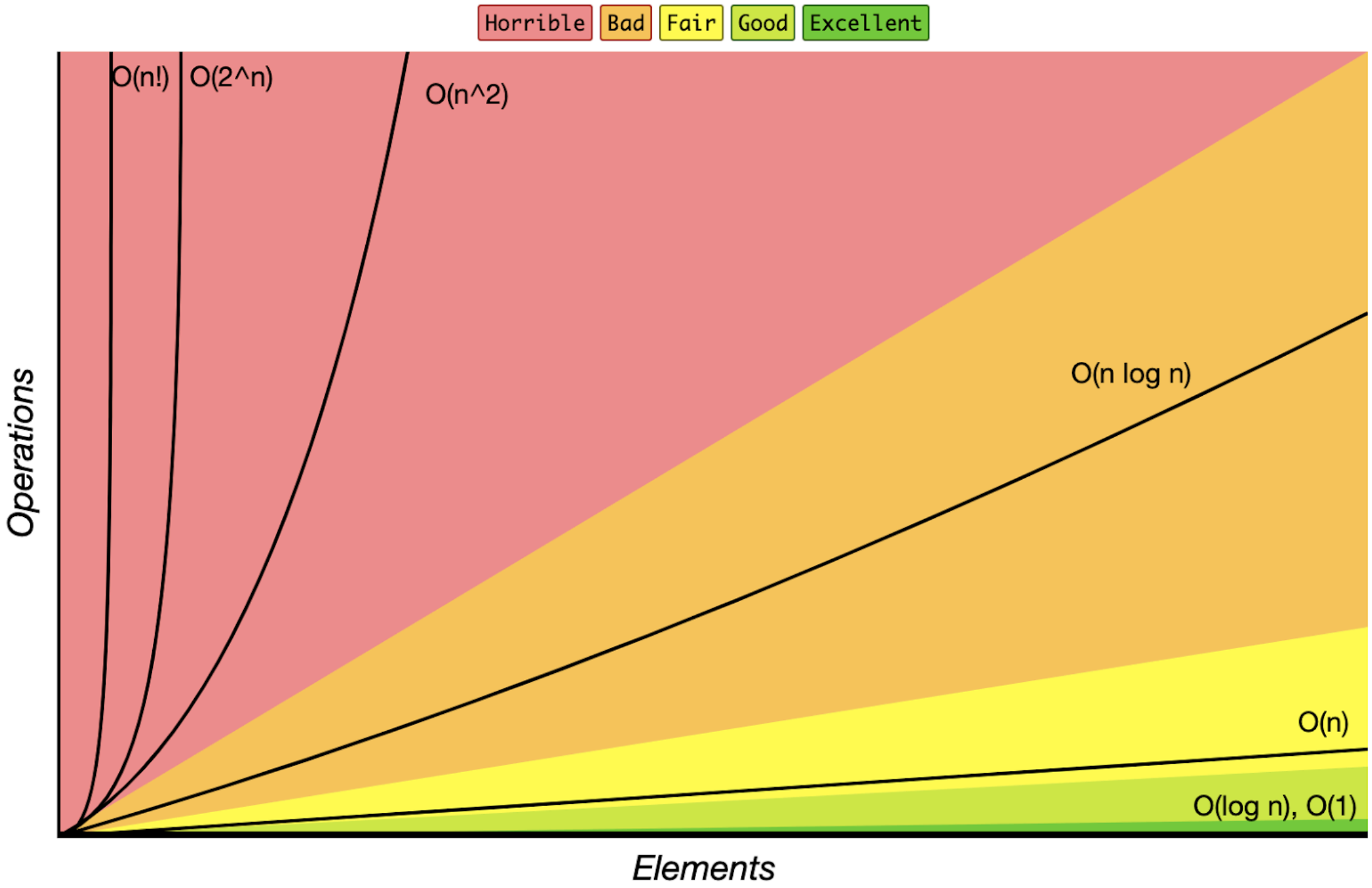
# Complexité en temps (le retour)

- La complexité en temps d'un algorithme est souvent exprimée en notation $\mathcal{O}$.
- Le but de l'analyse de la complexité est de pouvoir comparer les algorithmes entre eux.
- Il existe plusieurs classes de complexités, et la plupart des algorithmes que nous seront amenés à étudier appartiendra **nécessairement** à une de ces classes.

# Classes de complexité

- Les classes de complexité les plus courantes sont (dans l'ordre croissant) :
  - Constante $\mathcal{O}(1)$
  - Logarithmique $\mathcal{O}(\log n)$
  - Linéaire $\mathcal{O}(n)$
  - Linéarithmique $\mathcal{O}(n \log n)$
  - Polynomiale $\mathcal{O}(n^2)$, $\mathcal{O}(n^3)$, $\ldots$
  - Exponentielle $\mathcal{O}(2^n)$, $\mathcal{O}(3^n)$, $\ldots$
  - Factorielle $\mathcal{O}(n!)$

> **Note :** Ici log désigne le logarithme en base 2.


Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

# Analyse de la complexité d'un algorithme

- L'analyse *asymptotique* (lorsque la taille de l'entrée est grande) d'un algorithme détermine le temps d'exécution en notation $\mathcal{O}$.
- Avant d'essayer de déterminer la complexité d'un algorithme, **il est très important de savoir ce que fait l'algorithme**.
- On suit alors les étapes suivantes :
  i. Identifier le cas qui produit le plus d'opérations élémentaires (*le pire cas*).
  ii. Compter le nombre d'opérations élémentaires effectuées par l'algorithme.
  iii. Exprimer ce nombre d'opérations en notation $\mathcal{O}$.

# Est-ce qu'on doit compter toutes les opérations ?

# Est-ce qu'on doit compter toutes les opérations ?

Non, pas nécessairement :

- Comme on ne s'intéresse qu'à l'ordre de grandeur du temps d'exécution, on peut se contenter de compter **les opérations qui contribuent le plus au temps d'exécution**.
- Par exemple : le nombre d'affectations, le nombre de comparaisons, le nombre d'itération dans une boucle, etc.

> **Note :** Les opérations qui contribuent le plus au temps d'exécution sont celles qui permettent d'étudier le pire cas, s'il existe.

# Exemple 2: Complexité d'une boucle

```
Function: sum_array (T : array of integers, n: integer) → integer
Local variables:
    s : integer
    i : integer
begin
    s ← 0
    for i ← 1 to n do
        s ← s + T[i]
    end for
    return s
end
```

- Ici on considère que l'opération qui contribue le plus au temps d'exécution est l'opération `s ← s + T[i]`. **Elle s'exécute $n$ fois.**
- On a donc $n$ opérations de ce type. On peut donc dire que la complexité de `sum_array` est en $\mathcal{O}(n)$.

# Exemple 3: Complexité d'une boucle imbriquée

Analysons ensuite la fonction suivante qui prend en entrée une matrice carrée $n \times n$ et qui calcule la somme de tous ces éléments.

```
Fucntion: sum_matrix (m : array of integers, n: integer) → integer
Local variables:
    s : integer
    i, j : integer
begin
    s ← 0
    for i ← 1 to n do
        for j ← 1 to n do
            s ← s + m[i][j]
        end for
    end for
    return s
end
```

# Exemple 3: Complexité d'une boucle imbriquée

- L'opération qui contribue le plus au temps d'exécution de cette fonction est l'instruction `s ← s + m[i][j]`.
- Cette opération est répétée $n$ fois dans la boucle interne, et la boucle interne est répétée $n$ fois.
- On a donc $n \times n = n^2$ opérations de ce type.
- On peut donc dire que la complexité de `sum_matrix` est en $\mathcal{O}(n^2)$.

# Exemple 4: Complexité d'une conditionnelle

Ici nous avons une fonction qui prend en entrée un tableau d'entiers et qui affiche les éléments de ce tableau si la taille du tableau est paire.

```
Function: print_even_sized_array (T : array of integers, n: integer)
Local variables:
    i : integer
begin
    if n % 2 == 0 then
        for i ← 1 to n do
            write(T[i])
        end for
    else
        write("Array size is odd")
    end if
end
```

# Exemple 4: Complexité d'une conditionnelle

- L'opération qui contribue le plus au temps d'exécution de cette fonction est l'instruction `write(T[i])`.
- Si la taille du tableau est paire, cette opération est répétée $n$ fois, la complexité de la fonction est donc en $\mathcal{O}(n)$.
- Sinon elle est en $\mathcal{O}(1)$.
- On choisit la complexité la plus grande des deux branches, donc la complexité de `print_even_sized_array` est en $\mathcal{O}(n)$.

# En résumé

- De maniière générale pour les boucles imbriquées, on évalue la complexité de la boucle interne, puis on multiplie par le nombre d'itérations de la boucle externe.
- Pour les conditions, on garde la complexité la plus grande des branchements.
- De plus, si un algorithme appelle une fonction, alors on prend en compte la complexité de cette fonction.

# Application : Tri par sélection

```
Function: selection_sort (T : array of integers, n: integer)
Local variables:
    i, j, min_index : integer

begin
    for i ← 1 to n - 1 do
        min_index ← i
        for j ← i + 1 to n do
            if T[j] < T[min_index] then
                min_index ← j
            end if
        end for
        if min_index ≠ i then
            swap(T[i], T[min_index])
        end if
    end for
end
```

# Application : Tri par sélection

## 1. Comprendre l'algorithme

- L'algorithme trie un tableau d'entiers en parcourant le tableau et en échangeant les éléments pour les placer dans le bon ordre.
- À chaque itération de la boucle externe, pour chaque $i$:
  - on *cherche* l'indice du plus petit dans le reste du tableau (on itère sur les indices $i + 1$ à $n$ avec la variable $j$).
  - on l'*échange* avec l'élément courant (si l'indice du plus petit est différent de $i$).
- Le pire cas est quand le tableau est trié dans l'ordre décroissant.

# Application : Tri par sélection

## 2. Identifier les opérations qui contribuent le plus au temps d'exécution

- Dans cette version de l'algorithme, l'échange est effectué à chaque fois qu'on trouve un élément plus petit que l'élément courant.
- Mais dans tous les cas, on compare l'élément courant avec tous les éléments restants du tableau.
- L'opération qui contribue le plus au temps d'exécution est la comparaison `T[j] < T[min_index]`.

> **Note** : Pour l'ensemble des algorithmes de tri reposant sur des comparaisons, les comparaisons sont les opérations qui contribuent le plus au temps d'exécution.

# Application : Tri par sélection

*Le pire cas est quand le tableau est trié dans l'ordre décroissant.*

## 3. Compter le nombre d'opérations élémentaires

- Pour la boucle interne, on a :
  - Lorsque $i = 1$, on a $n - 1$ comparaisons.
  - Lorsque $i = 2$, on a $n - 2$ comparaisons.
  - $\dots$
  - Lorsque $i = n - 1$, on a $1$ comparaison.
- On a donc $n - 1 + n - 2 + \cdots + 1 = \frac{n(n-1)}{2}$ comparaisons.

# Application : Tri par sélection

## 4. Exprimer le nombre d'opérations en notation $\mathcal{O}$

- Si on note par $T(n)$, la complexité de l'algorithme `selection_sort`, on a $T(n) = \frac{n(n-1)}{2}$.
- En notation $\mathcal{O}$, on a $T(n) = \mathcal{O}(n^2)$.